Test Driven Development Témalabor

Csorba Kristóf

BME AUT
Department of Automation and Applied Informatics

Alapfeladat

Készítsen egy olyan Shop osztályt, aminek meg lehet adni termékeket és árakat (pl. 'A' és 54), egy kosár tartalmára (pl. "ABC") pedig meg tudja mondani az összesített árat.

Minden termék egy nagybetű, az ár pedig egész szám.

Például:

```
Shop.RegisterProduct('A', 10);
Shop.RegisterProduct('C', 20);
Shop.RegisterProduct('E', 50);
var price = Shop.GetPrice("ACEE");
     // 130-at ad vissza
```

Change Request Document (CRD)
P: Price (első adag)

CRD P01: mennyiségi kedvezmény

Ha egy termékből pl. legalább 5-öt veszek, a termék árából kapok 10% kedvezményt.

```
Shop.RegisterProduct('A', 10);
Shop.RegisterProduct('B', 100);
Shop.RegisterAmountDiscount('A', 5, 0.9);
     // 5 darabtól, 0.9-es szorzó
var price = Shop.GetPrice("AAAAAAB");
     // 6*10*0.9+100
```

CRD P02: 3-at fizet, 4-et vihet akció

Meg lehet adni olyan kedvezményt, hogy N darab vásárlása esetén M-et vihet ugyanazon az áron.

Például

```
Shop.RegisterProduct('A', 10);
Shop.RegisterProduct('E', 50);
Shop.RegisterCountDiscount('A', 3, 4);
    // 3 áráért 4-et vihet
var price = Shop.GetPrice("AAAAAEEE");
    // 5*10+3*50 helyett 4*10+3*50
```

CRD P03: Kombó kedvezmény

Pl. A, B és C külön-külön megvásárolva 20 lenne, de így együtt csak 16.

```
Shop.RegisterProduct('A', 10);
Shop.RegisterProduct('B', 20);
Shop.RegisterProduct('C', 50);
Shop.RegisterProduct('D', 100);
Shop.RegisterComboDiscount("ABC", 60);
var price = Shop.GetPrice("CAAAABB");
    // ABC+AAAB -> 60+3*10+20
```

CRD P04: Klubtagság

Ha egy kosárban szerepel a "t" betű (kisbetű, ami nem lehet termék), akkor az azt jelenti, hogy a pénztárnál lecsippantottak egy tagsági kártyát. Ekkor a végösszegből jár 10% kedvezmény.

CRD P05: Kombó klubtagoknak

A kombó kedvezményekre megadható, hogy az csak klubtagoknak jár.

CRD P06: User ID és pontgyűjtés

Ha a kosárban egy számjegy is szerepel (pl. '3'), akkor az a vásárló azonosító száma. Ekkor a program a kosár értékének 1%-át "supershop pontként" eltárolja. (Fájlba nem menti el, csak a memóriában tárolja most, így a program leállása után a pontok elvesznek.)

Ha egy kosárban szerepel a 'p' betű, akkor a vevő a "supershop" pontjaiból szeretne fizetni. A supershop pontjai levonódnak a végösszegből és csak a maradékot kell kifizetni.

Change Request Document (CRD)
P: Price (második adag)

CRD P07: Több kedvezmény egyszerre

Bár a megrendelő korábban elfelejtette megemlíteni, de természetesen az áruházban egyszerre több kedvezmény is érvényben van. Tehát lehet, hogy egyszerre 5 kombó kedvezmény, 4 mennyiségi kedvezmény és egy "3-at fizet 4-et vihet" akció is van. A vásárlók egy vásárlás alkalmával több kedvezményt is kihasználhatnak. És előfordulhat, hogy egy termék pl. mennyiségi kedvezményben és kombó kedvezményben is érintett. Ha mindkettő feltétele teljesül, akkor a viselkedés mindegy, de legyen egyértelműen leírva, hogy ilyenkor melyik érvényesül.

Vagyis: döntsétek el, mi történik ilyenkor és készítsetek rá teszteket, hogy a továbbiakban ez már ne változzon meg véletlenül. Tetszőleges, de ismert és kiszámítható legyen a viselkedés.

CRD P08: Több számjegyű User ID

A megrendelőnek már nem elég az egy számjegyű vásárlói azonosító. Helyette a kosár stringjában "v123" azt jelenti, hogy az 123-as vásárlóval van dolgunk. A 'v' után több számjegy is szerepelhet.

CRD P09: Akinek van User ID, az klubtag

A klubtagságot mostantól azzal jelzi a rendszer, hogy meg van adva a vásárló azonosítója. Vagyis a "supershop kártya" egyben tagsági kártya is. Korábban volt olyan feladat, hogy egy kedvezmény csak klubtagoknak jár.

Kérdés: mi a jobb? Módosítani a funkcionalitást (apró lépésekben!), vagy a kosár stringbe automatikusan User ID esetén kerüljön bele egy ‘t’ betű?

Tipp

- Ha egyszerre látszólag sok mindent kell módosítani, akkor először refactorálással érdemes előkészíteni a terepet.
- Ha egy csomó unit tesztet át kell írni, akkor mi less a TDD kis lépéseivel?
  - Először készítsd el az új működést (közben minden régi unit teszt zöld), majd
  - Töröld a feleslegessé vált kódrészleteket és a kapcsolódó unit teszteket. De tesztet csak akkor törölj, ha azért piros, mert a tesztelt kódot törölted, vagyis nem marad vissza kód, ami így teszteletlen less!

CRD P10: minden kedvezménynél kapcsolható, hogy csak klubtagoknak jár-e

Eddig nem volt elvárás, hogy minden kedvezményre meg lehessen adni, hogy csak klubtagoknak jár. Most már igen. (A kedvezmények regisztrációját végző metódusokban érdemes felvenni egy új paramétert erre, de hogy ne kelljen mindenhol átírni a korábbi tesztek esetében is, érdemes egy alapértelmezett értéket adni neki, hogy ha nincsen megadva, akkor nem csak klubtagokra érvényes.)

CRD P11: Egyedi kuponkódok

Lehessen regisztrálni a boltban kupon kódokat. Ha pedig egy kosárban az a kód szerepel, akkor kapjon kedvezményt. A számokból álló kupon kód a kosár stringjében egy 'k' után áll.

```
Shop.RegisterProduct('A', 10);
Shop.RegisterProduct('B', 20);
Shop.RegisterCoupon("112554", 0.9);
     // -10% kupon
var price1 = Shop.GetPrice("AABk112554");
     // 40*0.9
var price2 = Shop.GetPrice("AABk112554");
     // 40, mert már elhasználták a kupont
```

CRD P012: több termék egy vonalkóddal

A boltban megjelentek a pékáruk, amikhez a vásárló darabszámot is tartalmazó vonalkódot nyomtat.

Bizonyos termékek betűjele után megjelenik a darabszám. Például ‘A2B8C’, vagyis A-ból 2, B-ből 8 van a kosárban.

(Ez különösen kényelmes lesz a pénztárosnak, ha ő is meg tud adni a “lecsippantás” előtt egy darabszámot, de ebbe most nem megyünk bele.)

CRD P013: tömeg alapú árazás

A boltban megjelentek olyan termékek, amiket a vásárlónak le kell mérnie és a kapott vonalkódban a tömeg is benne van.

Bizonyos termékek betűjele után megjelenik a tömeg, grammban. Például ‘AB1200C’, vagyis a B termékből 1200 gramm van a kosárban.

(Olyan nem fordul elő, hogy egy terméknél darabszám és tömeg is megjelenik.)

Change Request Document (CRD)
D: Database

CRD D01: Leltár frissen tartása

Minden egyes vásárlás után a leltárban frissíteni kell a termékek darabszámát. A leltár itt egy interface, jelenleg egyetlen metódussal, aminek egy terméket és egy darabszámot lehet átadni.

CRD D02: Visszáru stornózása

Ha egy vásárló fizetés után visszahoz egy terméket, akkor visszakapja az árát. Annyit, amennyivel kevesebbet fizetett volna, ha nem veszi meg. (Ha például pont a megvásárlása miatt kedvezményeket kapott, akkor lehet, hogy kevesebbet kap vissza, mint a termék ára!)

A visszahozott termék visszakerül a raktárba. (Azzal most nem foglalkozunk, hogy hibás a termék vagy csak nem tetszett a vevőnek.)

Change Request Document (CRD)
C: Control

CRD C01: eseményvezérelt megoldás

Eddig a Shop egy stringben kapta meg a vásárlói kosár tartalmát. Alakítson ki egy metódust, aminek egyesével, egymás után lehet átadni a termékeket. Gyakorlatilag “a vonalkód leolvasó lecsippantott egy kódot” esemény eseménykezelője lesz ez.

Emellé kelleni fog egy “most pedig kérem a végösszeget” metódus, ami alkéri a kosár árát.

A string bemenetű kiértékelő metódus ezután felesleges.

Kérdés: érdemes törölni a string kiértékelő metódust?

CRD C02: pénztári mérleg

A bolt a pénztárhoz rakta a mérlegeket, nem a vásárlók nyomtatnak vele vonalkódot. Ha egy tömegben árazott termék kerül lecsippantásra, a program a mérlegtől, mint hardver perifériától kérje el a tömeget.

Tipp: a mérleg egy külön interface egy GetCurrentWeight() metódussal. Amint a vonalkód leolvasú eseménykezelője kap egy új terméket, az lekérdezi a mérlegtől a tömeget.

CRD C03: sikeres fizetés lejelentése

A főpénztáros szeretné mindig látni, ha valamit kifizettek. Ezért ad egy interfészt egy metódussal, aminek a sikeres fizetés után át kell adni a végösszeget.

A pénztáros a készpénzes fizetés megtörténtét egy külön nyomógombbal (új eseménykezelő meghívásával) jelzi.

CRD C04: kártyás és készpénzes fizetés

A sikeres fizetés nem csak készpénzzel történhet meg, hanem POS terminállal is. A pénztáros kap egy új nyomógombot (új esemény kezelővel a programunkban), amit ha megnyom, akkor a programunknak el kell küldenie az összeget a POS terminálnak (új interfész egyetlen metódussal, ami azonnal visszatér). A POS terminal utána meghívja egy metódusunkat, amivel jelzi, hogy a fizetés sikeres volt-e. Ha nem, a pénztáros majd újra megnyomja a készpénzes vagy terminális fizetés gombját.

CRD C05: Stornó funkció

A pénztáros sikeres fizetés előtt bármikor az egész kosárra vonatkozó munkamenetet törölheti a stornó gombbal (új eseménykezelő metódus).

Ekkor a fizetési folyamatot is meg kell szakítani, valamint a leltárból sem szabad kivenni a kosár tartalmát (ha már készült leltár funkció).

Change Request Document (CRD)
X: Extra further functions

CRD X01: Nap végi elszámolás

Nap végén lehessen kérni a rendszertől egy összesítést, hogy melyik termékből mennyi fogyott, valamint hogy ebből mennyi bevétel keletkezett. (A kedvezmények miatt a bevétel nem csak a termék ára szorozva a mennyiséggel.)

Tipp: ez elég összetett feladat, például kombó bónusz esetén. A nem specifikált kérdésekben a fejlesztő csapat hozzon tudatos döntést (ne úgy működjön, “ahogy sikerül”).

CRD X02: Kiskorúak

Kiskorúak bizonyos termékeket nem vásárolhatnak. Külön esemény jelezze a kiskorú vásárlót. Amennyiben olyan terméket vásárol, amit neki nem szabad (ezt termékenként lehessen megadni), akkor a rendszer jelezze a hibát egy “illegális vásárlás” jelzésére szolgáló interfészen keresztül.